

Attention Is All You Need

1. Introduction

- 기존 시퀀스 모델(RNN, LSTM, GRU)은 시간 순서대로 계산해야 하므로 병렬화가 불가능하고, 시퀀스 길이가 길어질수록 연산 효율과 메모리 문제가 발생함.
- 입력과 출력 간 거리가 멀어질수록 정보 전달 경로가 길어져 학습이 어려움.
- 순환(Recurrence)과 합성곱(Convolution)을 완전히 배제하고, 오직 **어텐션 (Attention)** 메커니즘만 사용하여 입력과 출력의 전역적 의존성을 학습하는 **트랜스포머(Transformer)** 구조를 제안함.

2. Background

- **기존 모델의 한계:** CNN 기반 모델(ByteNet, ConvS2S)은 병렬화는 가능하나, 거리에 따라 연산량이 증가하여 장거리 의존성 학습에 한계가 있음.
- **트랜스포머의 장점:**
 - 전체 의존성 계산을 상수 횟수($O(1)$)로 줄임.
 - 순차적 연산의 제약을 없애 병렬 처리에 최적화됨.
 - 시퀀스 길이(n)가 표현 차원(d)보다 작을 때 RNN보다 계산 복잡도가 낮아 더 빠름.

3. Model Architecture

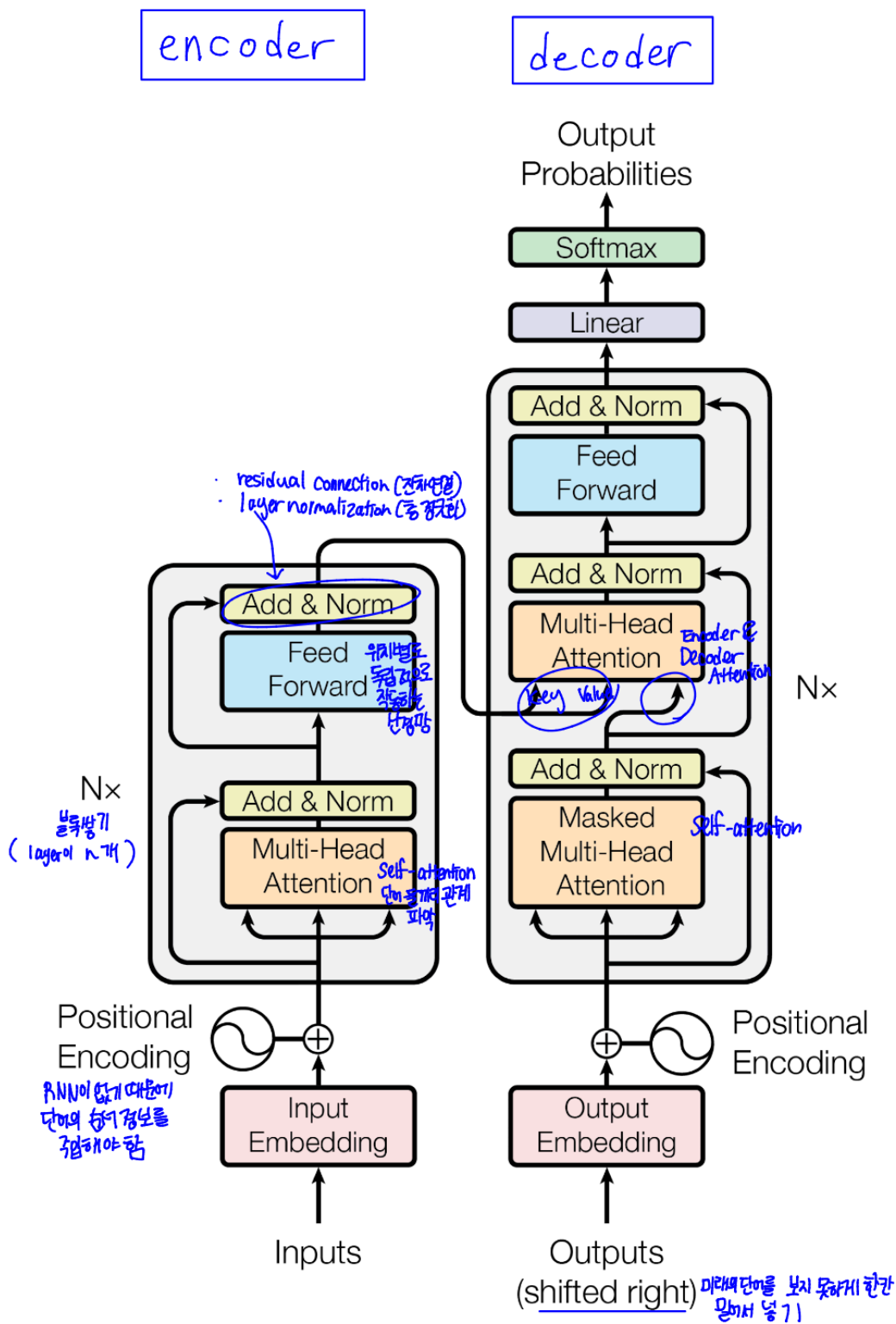


Figure 1: The Transformer - model architecture.

3.1 Encoder & Decoder Stacks

- **Encoder:**

- N=6개의 동일한 레이어로 구성.
- 각 레이어는 **Multi-Head Self-Attention**과 **Position-wise FFN**으로 이루어짐.
- 입력 전체를 한 번에 넣어 병렬적으로 처리함.
- **Decoder:**
 - N=6개의 레이어로 구성.
 - 인코더의 출력을 Key/Value로 참조하는 **Encoder-Decoder Attention** 서브 레이어가 추가됨.
 - **마스킹(Masking):** 자기회귀 속성을 유지하기 위해 현재 위치 이후의 토큰을 참조하지 못하도록 처리함.
- **공통 요소:**
 - **Residual Connection:** 정보 소실 방지.
 - **Layer Normalization:** 배치 정규화(Batch Norm) 대신 사용. 입력 샘플의 길이가 서로 다르므로 배치 통계량보다 샘플 내부의 정규화가 더 안정적임.

3.2 Attention

- **Scaled Dot-Product Attention:**
 - 수식: $\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V$
 - **스케일링(d_k):** d_k 값이 커질 때 내적 값이 커져 Softmax 기울기가 소실되는 것을 방지함.
-
- **Multi-Head Attention:**
 - Q,K,V를 h=8개의 서로 다른 공간으로 Linear Projection하여 병렬 수행.
 - 서로 다른 표현 부분 공간의 정보를 동시에 학습하여 성능을 극대화함.
 - **어텐션의 종류:**
 1. **Encoder Self-Attention:** 인코더 내의 모든 단어 간 관계 학습.
 2. **Decoder Self-Attention:** 디코더 내의 단어 간 관계 학습 (Masked).
 3. **Encoder-Decoder Attention:** 디코더(Query)가 인코더 출력(Key, Value) 정보를 참조.

3.3 Position-wise FFN

- 각 위치마다 개별적이고 동일하게 적용되는 두 번의 선형 변환과 ReLU 활성화 함수.
- 수식: $FFN(x) = \max(0, xW1 + b1)W2 + b2$

3.4 Embeddings and Softmax

- 학습된 임베딩을 사용하여 입력 및 출력 토큰을 d_{model} 차원의 벡터로 변환함.
- 디코더 출력을 예측된 다음 토큰 확률로 변환하기 위해 선형 변환(Linear Transformation)과 소프트맥스(Softmax) 함수를 사용함.
- 두 임베딩 레이어와 프리-소프트맥스(pre-softmax) 선형 변환이 동일한 가중치 행렬을 공유함.
- 임베딩 레이어에서는 공유된 가중치에 $\sqrt{d_k}$ 을 곱해줌.

3.5 Positional Encoding

- 순환이나 합성곱이 없으므로 시퀀스 순서 정보를 주입하기 위해 사인과 코사인 함수를 이용한 위치 인코딩을 입력 임베딩에 더함
- $PE_{(pos, 2i)} = \sin(pos/10000^{2i/d_{model}})$
- $PE_{(pos, 2i+1)} = \cos(pos/10000^{2i/d_{model}})$

4. Why Self-Attention

- **레이어당 총 계산 복잡도:** 시퀀스 길이(n)가 표현 차원(d)보다 작을 때 RNN보다 빠름
- **최소 순차 작업 수로 측정되는 병렬화 가능한 계산량:** 순차적으로 실행해야 하는 연산 수가 최소화되어 병렬 처리에 유리함
- **네트워크 내 장거리 의존성(long-range dependencies) 사이의 경로 길이:** 네트워크 내 임의의 두 위치 간 최대 경로 길이가 ($O(1)$)로 짧아 장거리 의존성 학습이 쉬움

5. Training

- WMT 2014 영어-독일어(4.5M 문장 쌍) 및 WMT 2014 영어-프랑스어(36M 문장 쌍) 데이터셋 사용.
- 8개의 NVIDIA P100 GPU를 사용. 베이스 모델은 12시간(100K steps), 빅(Big) 모델은 3.5일(300K steps) 동안 학습함.
- Adam optimizer와 Warmup 학습률 스케줄링을 적용함.

- 과적합 방지를 위해 Residual Dropout ($P_{drop} = 0.1$)과 Label Smoothing ($\epsilon_{ls}=0.1$) 기법을 사용함.

6. Results

Table 3: Variations on the Transformer architecture. Unlisted values are identical to those of the base model. All metrics are on the English-to-German translation development set, newstest2013. Listed perplexities are per-wordpiece, according to our byte-pair encoding, and should not be compared to per-word perplexities.

	N	d_{model}	d_{ff}	h	d_k	d_v	P_{drop}	ϵ_{ls}	train steps	PPL (dev)	BLEU (dev)	params $\times 10^6$
base	6	512	2048	8	64	64	0.1	0.1	100K	4.92	25.8	65
(A)				1	512	512				5.29	24.9	
				4	128	128				5.00	25.5	
				16	32	32				4.91	25.8	
				32	16	16				5.01	25.4	
(B)					16					5.16	25.1	58
					32					5.01	25.4	60
(C)	2									6.11	23.7	36
	4									5.19	25.3	50
	8									4.88	25.5	80
		256			32	32				5.75	24.5	28
		1024			128	128				4.66	26.0	168
			1024							5.12	25.4	53
			4096							4.75	26.2	90
(D)							0.0			5.77	24.6	
							0.2			4.95	25.5	
								0.0		4.67	25.3	
								0.2		5.47	25.7	
(E)		positional embedding instead of sinusoids								4.92	25.7	
big	6	1024	4096	16			0.3		300K	4.33	26.4	213

- WMT 2014 영어-독일어 태스크에서 28.4 BLEU를 기록하여 기존 앙상블 모델보다 2.0점 이상 높은 성능을 보임. 영어-프랑스어 태스크에서도 41.8 BLEU로 단일 모델 최고 기록을 경신함.
- 경쟁 모델들이 1019~1020 FLOPs 이상의 연산량을 소모한 데 비해, 트랜스포머는 1018 FLOPs 수준으로 **학습 비용을 획기적으로 절감함 (기존 대비 1/4 수준)**.
- 헤드 수(h)가 너무 적거나(Single-head) 많으면 성능이 하락했음. 키의 차원(d_k)을 줄이면 내적보다 정교한 호환성 함수의 필요성이 시사됨.

7. Conclusion

- **최초의 모델:** 순환(Recurrence)이나 합성곱(Convolution) 없이 오직 **어텐션** 메커니즘만으로 구축된 최초의 시퀀스 변환 모델임.

- **성과:** 기존 모델 대비 훨씬 빠른 훈련 속도와 더 높은 번역 품질(SOTA)을 달성함.
- **향후 계획:** 텍스트 이외의 이미지, 오디오, 비디오 등 다양한 모달리티로의 확장 및 생성 과정을 덜 순차적으로 만드는 연구를 계획함.

Neighbor Search와의 관련성 및 차이

- **Nadaraya-Watson Kernel Regression:** 1964년 제안된 non-parametric 회귀 모델로, 입력과 유사한 이웃 데이터의 가중 평균으로 예측값을 구함
- **Transformer Attention:** Nadaraya-Watson의 개념을 고차원 벡터 공간으로 확장하고, 데이터 간의 관계(Q, K, V)를 학습 가능한 행렬로 치환한 구조임.

- **Nadaraya-Watson Kernel Regression**

$$f(x) = \sum_{i=1}^n \frac{K(x, x_i)}{\sum_{j=1}^n K(x, x_j)} y_i$$

- x (**입력**): 새로운 쿼리 포인트.
- x_i (**위치**): 기존 데이터의 인덱스/키.
- y_i (**값**): 실제 관측된 데이터 값.
- K (**커널**): 두 점 사이의 유사도를 측정하는 함수 (예: Gaussian Kernel)

- **Transformer Attention**

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V$$

- Q (**Query**): 정보를 찾고자 하는 주체 벡터.
- K (**Key**): 각 데이터가 가진 특징 벡터.
- V (**Value**): 추출하고자 하는 실제 정보 벡터.
- **Softmax:** 유사도(QK^T)를 확률 분포로 변환하는 커널 역할 수행.

- **details**

- **스케일링(Scaling) 유무:** Nadaraya-Watson은 커널 자체의 대역폭(Bandwidth)으로 밀도를 조절. Transformer는 d_k 가 커질 때 Softmax 기울기 소실을 막기 위해 $\sqrt{d_k}$ 로 나누는 스케일링 과정을 추가함.
- **Multi-Head 구조:** Nadaraya-Watson은 단일 유사도 척도만 사용함. Transformer는 $h = 8$ 개의 헤드를 통해 서로 다른 측면(Syntactic, Semantic 등)에서 정보를 동시에 검색함.

- **위치 정보 처리:** Nadaraya-Watson은 입력값(x) 자체가 위치 정보를 포함함. Transformer는 순서 정보가 없는 Attention의 특성을 보완하기 위해 별도의 Positional Encoding을 더해줌